Project - Low Level Design

on

CI/CD Pipeline for Autonomous Energy Researcher Agent

Course Name: Datagami Skill Based Course

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	Pranjal Sharma	EN22CS301731
2	Riya Patidar	EN22CS301813
3	Roshni Patidar	EN22CS301825
4	Srashti Bhadoriya	EN22CS301982
5	Ritik Patidar	EN22CS301809

Group Name: 10D7

Project Number: DO-33

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Shyam Patel

Academic Year: 2026

Table of Contents

1 **Introduction**
1.1 Scope
1.2 Intended Audience
1.3 System Overview

2 **System Design**
2.1 Application Architecture
2.2 Process Flow
2.3 Information Flow
2.4 Database Design
2.5 Containerization (Docker)
2.6 Key Design Considerations
2.7 API Endpoints
2.8 Technology Stack
2.9 Error Handling
2.10 Security

1. Introduction

1.1 Scope

This document describes the low-level technical design of the EnergyMind AI Agents system. It explains the internal architecture, data flow, components, database design, and APIs used in the system.

1.2 Intended Audience

This document is intended for developers, system architects, and QA engineers who will implement, maintain, and test the system.

1.3 System Overview

EnergyMind AI Agents is an autonomous research assistant for the energy industry. It uses a multi-agent workflow orchestrated by LangGraph to process user queries, search the web using Tavily, analyze results using Groq LLM (Llama 3.1), and generate structured research reports. The system consists of a FastAPI backend, a Streamlit frontend, and a SQLite database.

2. System Design

2.1 Application Architecture

The system follows a client–server architecture.

The Streamlit frontend provides the user interface and handles authentication using JWT tokens.
The FastAPI backend manages API requests and triggers the AI workflow.
The LangGraph layer coordinates multiple AI agents responsible for research, analysis, and report generation.
SQLite is used to store users, chat history, and cached research results.

2.2 Process Flow

The research process follows a multi-agent workflow.

First, the Gatekeeper checks whether the user query is related to energy. If the query is irrelevant, the process stops.

If the query is valid, the Researcher agent retrieves information using the Tavily search API.

The Analyst agent processes the retrieved data and extracts insights such as trends, risks, and opportunities.

The Writer agent generates a structured report from the analysis.

The Reviewer agent checks the report quality. If the report is not satisfactory, the workflow returns to the Writer for revision.

Once approved, the Suggester agent generates follow-up questions for the user.

2.3 Information Flow

The user submits a query through the Streamlit interface. The request is sent to the FastAPI backend using HTTPS.

The backend verifies the JWT token and initializes a LangGraph state for the query.

Agent nodes may call external tools such as Groq LLM for reasoning and Tavily for web search.

After the workflow completes, the generated report is stored in the SQLite database and returned to the frontend as a JSON response.

2.4 Database Design

The database contains three logical datasets.

User information is stored with username, email, and hashed password.

Chat history records the user query, AI response, user ID, and timestamp.

A knowledge base cache stores previously generated reports for repeated queries to reduce API usage and improve performance.

2.5 Containerization (Docker)

The system is containerized using Docker for consistent deployment.

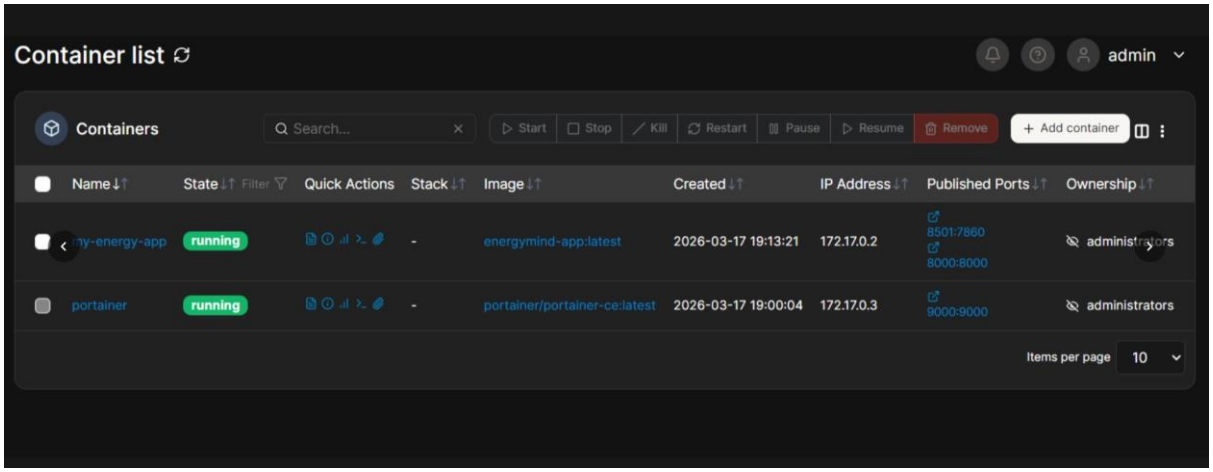
The container uses the Python 3.10 slim image. Dependencies are installed separately to optimize build caching.

A startup script launches both the FastAPI backend and Streamlit frontend.

The backend runs on port 8000 and the frontend runs on port 7860. During deployment on AWS EC2, these ports are mapped to the host to allow external access.

API keys such as Groq and Tavily credentials are provided through environment variables instead of hardcoding them in the application.

Portainer is deployed on port 9000 to monitor containers, inspect logs, and manage Docker resources.



2.6 Key Design Considerations

The system is modular, allowing individual agent nodes to be updated independently.

A reviewer–writer feedback loop ensures the quality of generated reports.

Docker-based deployment enables easy scaling on cloud platforms such as AWS EC2 or ECS.

Caching research results in SQLite reduces repeated API calls and improves performance.

2.7 API Endpoints

The backend exposes several endpoints.

The /signup endpoint registers a new user.

The /token endpoint authenticates a user and returns a JWT token.

The /research endpoint triggers the AI research workflow.

The /history endpoint returns the recent research queries of the authenticated user.

The /me endpoint returns the profile details of the logged-in user.

2.8 Technology Stack

The system is developed using Python 3.10.

FastAPI is used for the backend API. Streamlit is used for the frontend interface. LangGraph and LangChain manage the agent workflow. SQLite with SQLAlchemy ORM is used for data storage. Tavily API is used for web search. Groq (Llama 3.1) provides LLM capabilities. Docker and AWS EC2 are used for deployment.

2.9 Error Handling

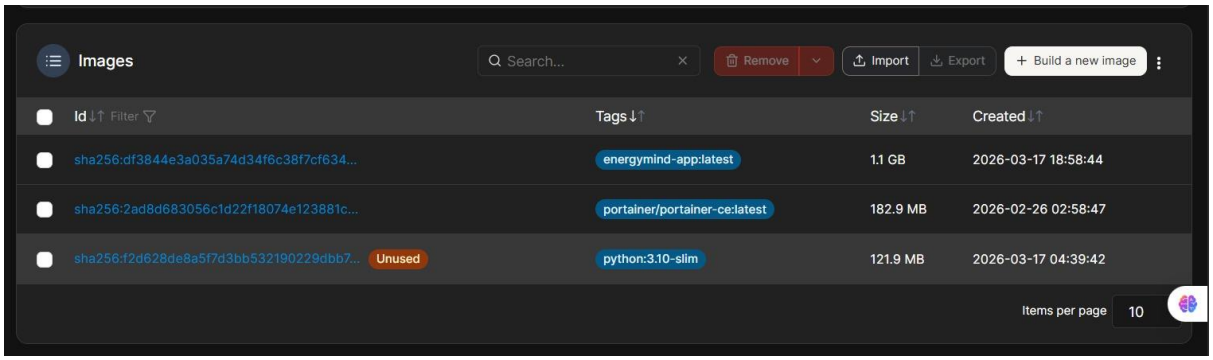
The system handles failures from external APIs and returns user-friendly error messages.

LangGraph MemorySaver maintains workflow state during execution.

The reviewer–writer loop acts as a logical retry mechanism to improve report quality.

2.10 Security

JWT authentication is used for secure API access.

CORS middleware protects against unauthorized cross-origin requests.

API credentials are stored as Docker environment variables instead of being embedded in the source code.